# Trusted Execution Environments

## Integration Opportunities for r*Stack & EncryptID

Prepared February 15, 2026

Confidential — Internal Use

# Contents

# 1 Executive Summary

Trusted Execution Environments (TEEs) allow sensitive computation to run on hardware that even the server operator cannot inspect. They are the missing piece for making the r*Stack's "zero-trust server" promise cryptographically enforceable rather than policy-based.

> **Key finding:** The highest-ROI integration point is **EncryptID's guardian recovery flow**, where TEEs ensure that a reconstructed master key is never visible to the server. This single change strengthens the security story for every r*Stack application that depends on EncryptID.

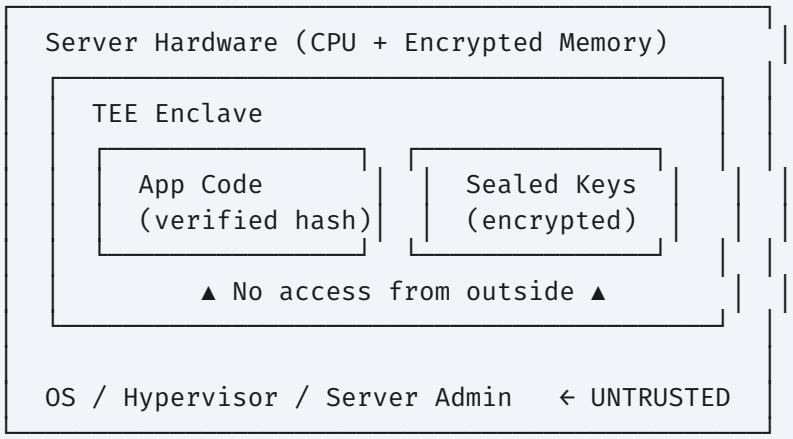
Five TEE platforms were evaluated. **Phala Network** and **Marlin Oyster** emerge as the top contenders for the r*Stack because they accept standard Docker containers, offer JavaScript/TypeScript SDKs, and provide on-chain attestation — aligning with the existing Bun/Node.js + EVM stack without requiring a rewrite.

# 2 What Are TEEs?

A TEE is a hardware-isolated region of a processor where code and data are protected from all other software on the machine — including the operating system, hypervisor, and anyone with physical access to the server. TEEs provide three guarantees:

| Guarantee | What It Means |
|---|---|
| **Confidentiality** | Memory inside the TEE is encrypted; even a root user or hypervisor cannot read it. |
| **Integrity** | Code inside the TEE cannot be tampered with. If modified, the TEE refuses to run. |
| **Attestation** | A remote party can cryptographically verify exactly which code is running inside the TEE, and that it is running on genuine hardware. |

Hardware TEE Isolation Model




The practical impact: you can run a server that processes sensitive data (private keys, votes, documents) while proving to users that you yourself cannot read their data. This is the core value proposition for EncryptID and the r*Stack.

# 3 Top Contender Platforms

## 3.1 Phala Network

| Attribute | Details |
|---|---|
| TEE Hardware | Intel SGX (current), Intel TDX (roadmap) |
| Deployment Model | Decentralized network of TEE workers; deploy Docker images |
| SDK / Language | **JavaScript/TypeScript SDK**, Rust (Phat Contracts) |
| Attestation | On-chain attestation on Phala (Substrate) + cross-chain bridges |
| Blockchain | Polkadot ecosystem, EVM-compatible via bridges |
| Maturity | Production since 2022; 30,000+ TEE workers in the network |
| Cost Model | Pay per compute (PHA token); competitive with serverless |
| Key Feature | **Phat Contracts** — off-chain confidential computation callable from any EVM chain |

> **Why it fits:** Phala's JS/TS SDK and Docker support mean you can wrap existing EncryptID or rSpace services as Phat Contracts with minimal changes. On-chain attestation lets smart contracts verify the TEE output trustlessly.

### 3.2 Marlin Oyster

| Attribute | Details |
|---|---|
| TEE Hardware | AWS Nitro Enclaves (current), Intel TDX (expanding) |
| Deployment Model | Decentralized TEE-as-a-service; deploy standard Docker images |
| SDK / Language | Any language — runs unmodified Docker containers |
| Attestation | On-chain attestation verification (Ethereum, Arbitrum, etc.) |
| Blockchain | Chain-agnostic; EVM-native attestation contracts |
| Maturity | Production; used by Kalypso (ZK proving) and other protocols |
| Cost Model | POND/MPOND token staking; competitive hourly rates |
| Key Feature | **HTTP Gateway** — TEE services accessible via standard HTTP, easy to integrate |

> **Why it fits:** Lowest friction path. Dockerize your existing Node.js/Bun service, deploy to Oyster, and it runs in a TEE. The HTTP gateway means your frontend makes normal API calls — no TEE-specific client code needed.

### 3.3 Oasis Sapphire

| Attribute | Details |
|---|---|
| TEE Hardware | Intel SGX |
| Deployment Model | Confidential EVM chain — deploy Solidity contracts with encrypted state |
| SDK / Language | Solidity + **ethers.js/viem-compatible** |
| Attestation | Built into the chain's consensus; transparent to developers |
| Blockchain | EVM-compatible L1 (Oasis Network) |
| Maturity | Production mainnet since 2022 |
| Cost Model | ROSE token for gas; low fees |
| Key Feature | **Confidential EVM** — standard Solidity, but contract state is encrypted |

> **Why it fits:** If rWallet or rVote needs on-chain confidential logic (private votes, hidden transaction amounts), Sapphire lets you write standard Solidity where the contract state is automatically encrypted. Works with existing ethers.js/viem tooling.

### 3.4 AWS Nitro Enclaves

| Attribute | Details |
|---|---|
| TEE Hardware | Custom Nitro hardware isolation |
| Deployment Model | Enclave inside an EC2 instance; communicates via vsock |
| SDK / Language | C SDK (primary); Rust, Python wrappers; Node.js via REST bridge |
| Attestation | Nitro Attestation Document (integrates with AWS KMS) |
| Blockchain | N/A (centralized cloud) |
| Maturity | Production GA since 2020; battle-tested |
| Cost Model | No extra cost beyond EC2 instance price |
| Key Feature | **KMS integration** — KMS only releases keys after verifying enclave attestation |

> **Trade-off:** Most mature and battle-tested, but centralized (AWS). Good for bootstrapping TEE capabilities quickly; can migrate to decentralized platforms later. No on-chain attestation without a bridge.

### 3.5 Gramine (Intel SGX LibOS)

| Attribute | Details |
|---|---|
| TEE Hardware | Intel SGX |
| Deployment Model | Self-hosted; wraps unmodified Linux apps in SGX enclaves |
| SDK / Language | Any language — runs Node.js, Python, Go, etc. unmodified |
| Attestation | Intel DCAP (self-hosted attestation infrastructure) |
| Blockchain | N/A (bring your own) |
| Maturity | Production; used by Signal for contact discovery |
| Cost Model | Free / open-source; requires SGX-capable hardware |
| Key Feature | **Zero code changes** — write a manifest file, your existing app runs in SGX |

> **Trade-off:** Maximum control and zero cloud dependency, but requires SGX-capable hardware. Best for self-hosted infrastructure if TEE hardware is available on Netcup RS 8000.

### 3.6 Platform Comparison Matrix

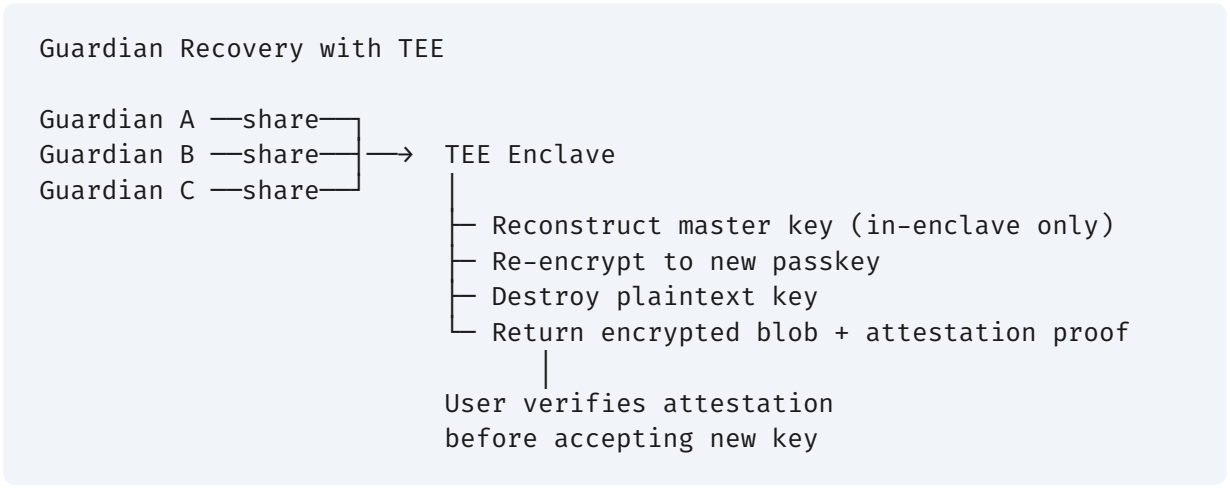
| Criteria | Phala | Marlin | Oasis | Nitro | Gramine |
|---|---|---|---|---|---|
| JS/TS Support | Native SDK | Via Docker | ethers.js | Via bridge | Via LibOS |
| Docker Deploy | Yes | Yes | N/A | EIF images | GSC tool |
| On-Chain Attestation | Yes | Yes | Built-in | No | No |
| EVM Compatible | Via bridge | Native | Native | N/A | N/A |
| Decentralized | Yes | Yes | Yes | No (AWS) | Self-hosted |
| Integration Effort | Low-Med | Low | Low-Med | Medium | Medium |
| Best For | Off-chain compute | Backend services | On-chain privacy | Key mgmt | Self-hosted |

## 4 Application to r*Stack & EncryptID

### 4.1 EncryptID — Guardian Recovery Enclave [HIGH IMPACT]

**The problem:** When 3-of-5 guardians submit recovery shares, the master key must be reconstructed somewhere. Today, that reconstruction happens on a server, meaning the server operator momentarily has access to the user's master key.

**TEE solution:** Run the Shamir secret sharing reconstruction inside a TEE enclave. The key is reconstructed, used to re-encrypt to the new passkey, and immediately discarded — all inside hardware the server operator cannot inspect.




**Recommended platform:** Marlin Oyster (Dockerize existing recovery service) or Phala (Phat Contract callable from EVM).

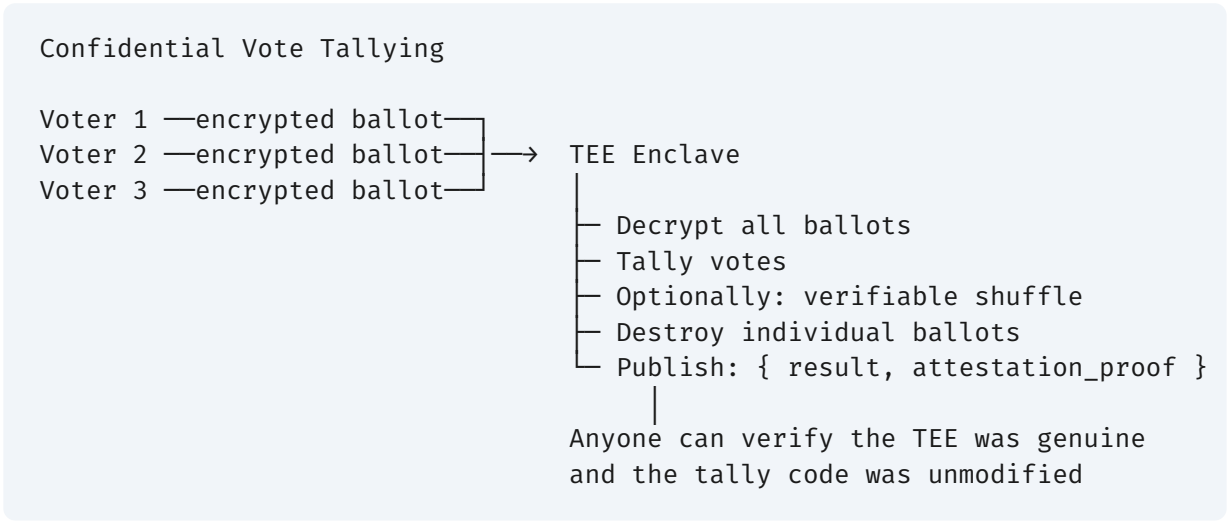
**Additional EncryptID TEE uses:**

- **SSO JWT signing key** — sealed inside TEE; even a compromised server can't extract it
- **Turnkey attestation chain** — verify that Turnkey's passkey signing genuinely runs in a TEE
- **Session token validation** — cross-app SSO broker runs in TEE for zero-trust identity

### 4.2 rVote — Confidential Tallying [HIGH IMPACT]

**The problem:** Vote secrecy requires that individual ballots are never visible to the server. Currently, either votes are visible to the backend during tallying, or complex homomorphic encryption is needed.

**TEE solution:** Encrypted ballots are submitted to a TEE, which decrypts, tallies, and publishes only the aggregate result along with an attestation proof. Nobody — not even the server admin — sees individual votes.

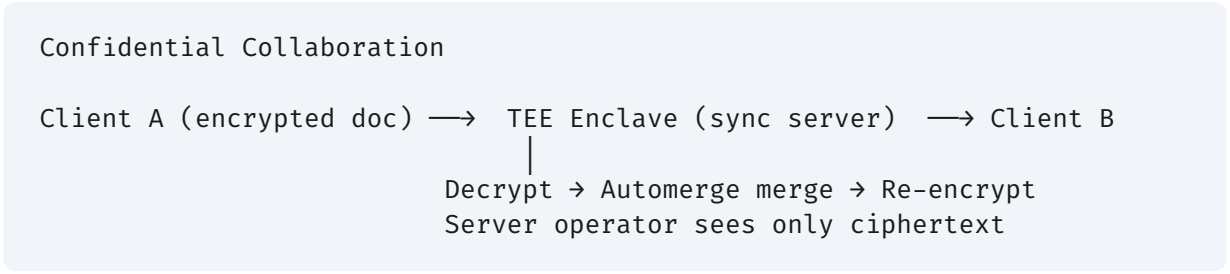



**Recommended platform:** Oasis Sapphire for on-chain governance (confidential Solidity), or Marlin Oyster for off-chain tallying with on-chain attestation.

### 4.3 rSpace — Confidential CRDT Merge [MEDIUM-HIGH IMPACT]

**The problem:** The collaboration server needs to merge Automerge CRDTs, which means it processes document content. Users must trust the server operator not to read their documents.

**TEE solution:** The sync server runs inside a TEE. Documents are encrypted client-side, decrypted inside the TEE for merging, and re-encrypted before being sent to other participants. The server processes documents it literally cannot read.




**Recommended platform:** Marlin Oyster (Docker container) or Gramine (if self-hosted with SGX hardware).

### 4.4 rFiles — Searchable Encryption [MEDIUM IMPACT]

**The problem:** Files are encrypted client-side (AES-256-GCM), but users cannot search across encrypted files without downloading and decrypting everything locally.

**TEE solution:** A TEE-based indexing service decrypts files inside the enclave, builds a search index, and responds to encrypted queries — all without the server seeing file contents.

**Additional use:** Proxy re-encryption for file sharing — the re-encryption key is generated inside a TEE, so the server never holds a key capable of decrypting the file.

**Recommended platform:** Phala (Phat Contract for search computation) or Marlin Oyster.

### 4.5 rWallet — Policy Enforcement [MEDIUM IMPACT]

**The problem:** Spending limits and transaction policies are enforced by the server. A compromised server could bypass these limits.

**TEE solution:** The policy engine (spending limits, whitelists, rate limits) runs inside a TEE. The paymaster signing key is sealed to the enclave. Even if the server is compromised, an attacker cannot bypass spending policies or extract the paymaster key.

**Recommended platform:** Oasis Sapphire (on-chain policy as confidential contract) or Marlin Oyster (off-chain policy engine).

## 5 Recommended Roadmap

| Phase | Scope | Platform | Effort | Impact |
|---|---|---|---|---|
| PHASE 1 | **EncryptID Recovery Enclave** Guardian key reconstruction + SSO signing key in TEE | Marlin Oyster or Phala | 2–4 weeks | **High — secures the foundation for all r*apps** |
| PHASE 2 | **rVote Confidential Tallying** Encrypted ballots → TEE tally → attested results | Oasis Sapphire or Marlin | 3–5 weeks | **High — strong differentiator for governance** |
| PHASE 3 | **rSpace Confidential Sync** CRDT merge inside TEE with remote attestation | Marlin Oyster | 4–6 weeks | **Medium-High — zero-trust collaboration** |
| PHASE 4 | **rFiles Searchable Encryption + rWallet Policy Engine** TEE-based search index and sealed policy enforcement | Phala / Marlin | 4–6 weeks each | **Medium — defense-in-depth** |

> **Phase 1 is the lynchpin.** Since every r*Stack app authenticates through EncryptID, hardening the identity layer with TEEs automatically raises the security floor for rSpace, rVote, rFiles, rWallet, and all future apps.

## 6 Infrastructure Considerations

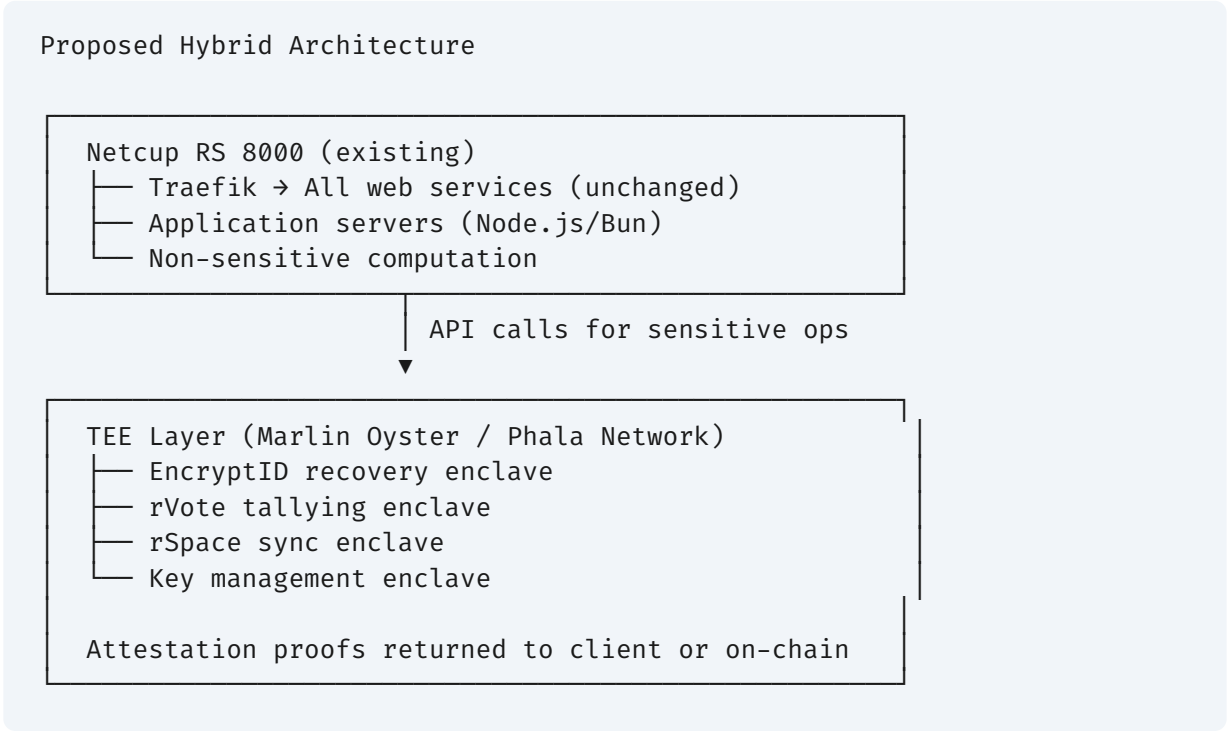
### Netcup RS 8000 — Hardware TEE Availability

The Netcup RS 8000 G12 Pro server should be checked for TEE support. Most hosting providers do not expose TEE features (SGX/SEV) in their firmware configuration. Run the following to verify:

| Check | Command |
|---|---|
| SGX support | `grep -i sgx /proc/cpuinfo` |
| AMD SEV support | `dmesg \| grep -i sev` |
| CPU model | `lscpu \| head -20` |

**If hardware TEE is unavailable** (likely), use Marlin Oyster or Phala Network for TEE workloads. These are decentralized TEE networks — you deploy Docker containers to their infrastructure, which runs on verified TEE hardware. This avoids needing your own TEE-capable server.

## Hybrid Architecture




## Netcup vGPU + NVIDIA Confidential Computing

The Netcup vGPU servers (RS 2000/4000 vGPU with NVIDIA H200) support NVIDIA Confidential Computing. If a vGPU is acquired, it could serve double duty: GPU inference and confidential AI computation. This is relevant if the r*Stack incorporates AI features that process sensitive user data.

## 7 Risks & Mitigations

| Risk | Severity | Mitigation |
|---|---|---|
| **Side-channel attacks on SGX** (cache timing, speculative execution) | Medium | Use platforms with latest microcode patches. Prefer TDX/SEV-SNP (smaller attack surface) where available. Defense-in-depth with application-layer encryption. |
| **Platform lock-in** (dependency on Phala/Marlin) | Medium | Design TEE services as standard Docker containers with a thin attestation wrapper. Switching platforms = redeploying the same container + updating attestation verification. |
| **Availability of decentralized TEE networks** | Low-Med | Fallback to AWS Nitro Enclaves as a centralized backup. Design recovery flow to degrade gracefully if TEE is temporarily unavailable. |
| **Development complexity** | Low | Both Marlin and Phala accept standard Docker images. The enclave code is the same code you'd write without TEEs — the platform handles hardware isolation transparently. |
| **Token economics** (PHA, POND/MPOND for usage) | Low | Budget for token acquisition. Costs are competitive with traditional cloud. Evaluate stablecoin payment options as platforms mature. |
| **Performance overhead** (enclave transitions, memory encryption) | Low | 2–10% overhead for VM-level TEEs (TDX/SEV-SNP). Batch sensitive operations to minimize enclave transitions. Benchmark early in Phase 1. |

> **Bottom line:** TEEs are the most practical path to making r*Stack's privacy claims verifiable. Start with EncryptID (Phase 1), prove the pattern, and extend to the rest of the stack. Phala and Marlin make this achievable without new hardware or a major rewrite.